

Identity & Authorization

Module Reference — Multi-Tenant Cloud Platform (v2)

This document explains the **Identity domain** (who are you?) and the **Authorization domain** (what can you do?) — with clear flowcharts, state machines, real examples using Acme Corp, alice, and bob.

Domain	Responsibility	Phase
Identity	Users, Keycloak realms, sessions, service accounts, API keys	Phase 1
Authorization	Permissions, roles, role bindings, RBAC evaluation	Phase 2

PART 1 — IDENTITY DOMAIN

The Identity domain answers: **"Who is this person and how do we prove it?"** It manages every type of principal — humans logging in, robots using API keys, and platform operators on the admin API. It integrates with **Keycloak**, the platform's authentication engine.

1.1 What Is a Principal?

A principal is any entity that calls the API. Three types exist:

Principal	Example	Authenticates via	Scope
User (human)	alice@corp.com	Email + password via Keycloak	Platform or org realm
ServiceAccount	sa-deploy-prod	API key (hashed, shown once)	One specific tenant + org
Platform Operator	platform-admin	Keycloak operator client (VPN only)	Operator org only

1.2 Keycloak — Three Realm Types

Keycloak organises users into **realms** — isolated login spaces. The platform uses three distinct realm types depending on who is logging in.

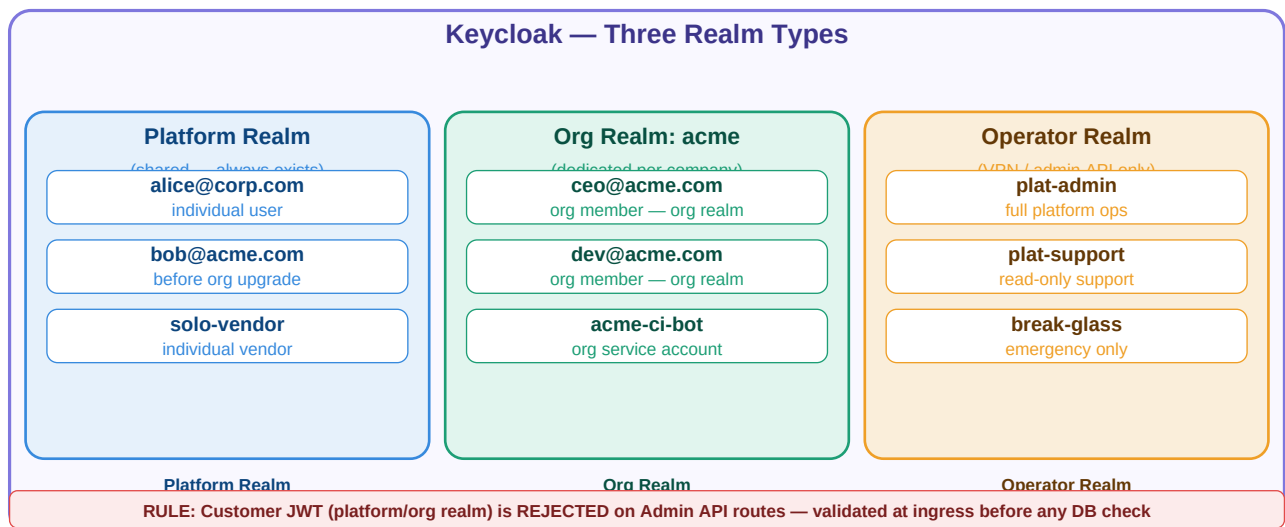


Figure 1 — Three Keycloak realm types: platform (shared), org (dedicated per company), operator (VPN only)

Realm	Who uses it	Created when	Stored on
Platform realm	All individual users + solo vendors	Platform deployment (once)	Identity config
Org realm	Company (org_type=organization) members	During org registration	Organization.keycloak_realms_ref
Operator realm	Platform ops team — VPN only	Platform deployment	Separate Keycloak client

■ Customer JWTs are NEVER accepted on the Admin API — validated at ingress via the token "iss" (issuer/realm) claim before any database check.

1.3 Identity Entities

User

A User is a human registered on the platform. Every User has a **primary_keycloak_subject** — Keycloak's unique ID for them (like a passport number) — which the platform maps to its own stable **user_id**.

Field	What it stores	Example
id	Platform UUID (stable forever)	user-uuid-alice-001
email	Unique across the whole platform	alice@corp.com
display_name	Human-readable label	Alice Smith
status	Current lifecycle state	active
primary_keycloak_subject	Keycloak's ID for this user	kc-sub-a8f3b2c1
primary_realm_ref	Which realm they authenticate through	platform-realm

User Lifecycle State Machine

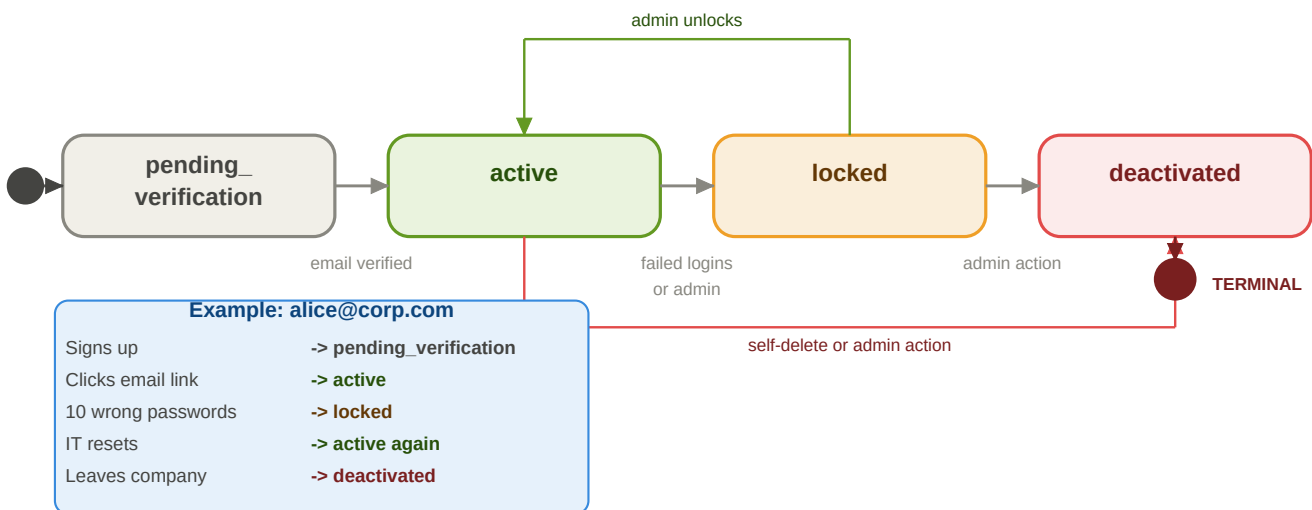


Figure 2 — User state machine: forward-only with one reversible recovery arc (locked -> active)

State	Meaning	Can log in?
pending_verification	Registered — email not yet confirmed	NO
active	Normal operating state — all permitted actions	YES
locked	Too many failed logins or admin blocked	NO — sessions invalidated
deactivated	Permanently disabled — data retained for audit	NO — TERMINAL

■ **deactivated is TERMINAL** — no reactivation. A new account must be created. The old record is kept for audit and compliance purposes.

ServiceAccount and ApiKey

A **ServiceAccount** is a non-human principal for automation (CI/CD pipelines, scripts). It always belongs to one specific tenant and org. An **ApiKey** is the actual secret it uses to authenticate.

Entity	Key rule	Example
ServiceAccount	Scoped to one tenant_id + organization_id — cannot act outside	sa-deploy-prod in acme-prod
ApiKey	Shown ONCE at creation — platform stores only key_hash (bcrypt)	ak_abc12345_xyz...
ApiKey scopes	Limits what the key can do — least privilege	["resource.read", "resource.create"]
ApiKey expiry	Optional expires_at — prefer short-lived for CI	2024-12-31T23:59:59Z

Session

Created when a human logs in. Allows refresh-token-based session extension without re-entering password. Revoking a session forces re-login after the current JWT expires.

State	Meaning	Recovery
active	Login valid — refresh token works	N/A
revoked	Logout or admin-killed — refresh token invalid	User must log in again
expired	TTL elapsed — too old to refresh	User must log in again

1.4 Workflow A — User Registration

When someone signs up: Identity creates the user in Keycloak, then fires a **UserRegistered** event. The Tenant domain reacts to create the default individual organization and tenant automatically.

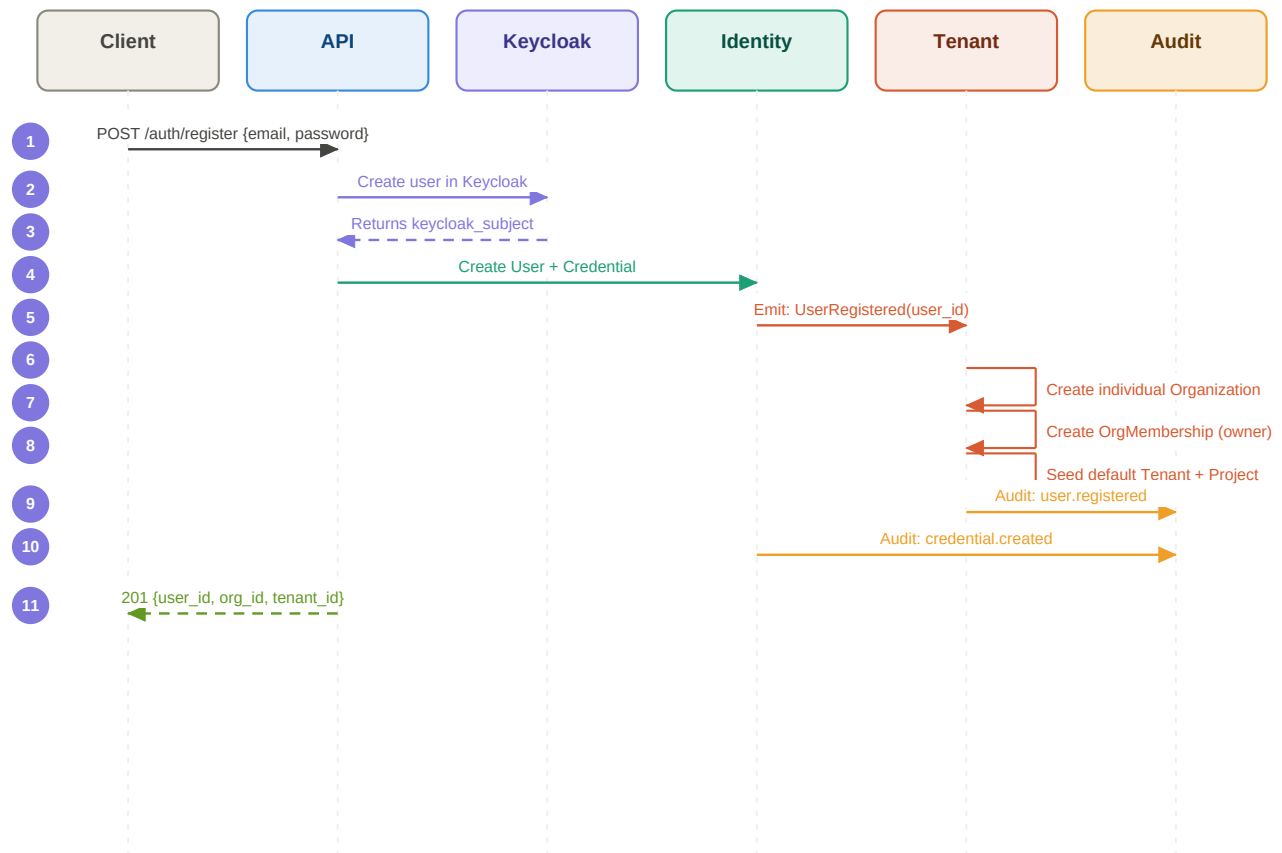


Figure 3 — User registration: 11 steps across 6 actors. Dashed arrows = return responses.

#	Actor	Action	Output
1	Client	POST /auth/register {email, password}	Request received
2	API	Create user in Keycloak platform realm	Keycloak user created
3	Keycloak	Returns keycloak_subject	kc-sub-a8f3b2c1
4	Identity	Create User row + Credential row in DB	user_id = user-uuid-alice-001
5	Identity	Emit UserRegistered(user_id) via outbox	Event written to outbox
6	Tenant	Create Organization (org_type=individual)	org-uuid-jane-001
7	Tenant	Create OrganizationMembership (owner role)	alice = owner
8	Tenant	Seed default Tenant + Project	alice-default tenant ready
9	Audit	Log: user.registered	Audit trail entry
10	Identity	Log: credential.created	Audit trail entry
11	API	Return 201 {user_id, org_id, tenant_id}	Client receives context

1.5 Workflow B — Organization Registration (Keycloak Realm Creation)

When a user upgrades from individual to a company organization, a dedicated Keycloak realm is provisioned for that company. All company employees then log in through their own realm.

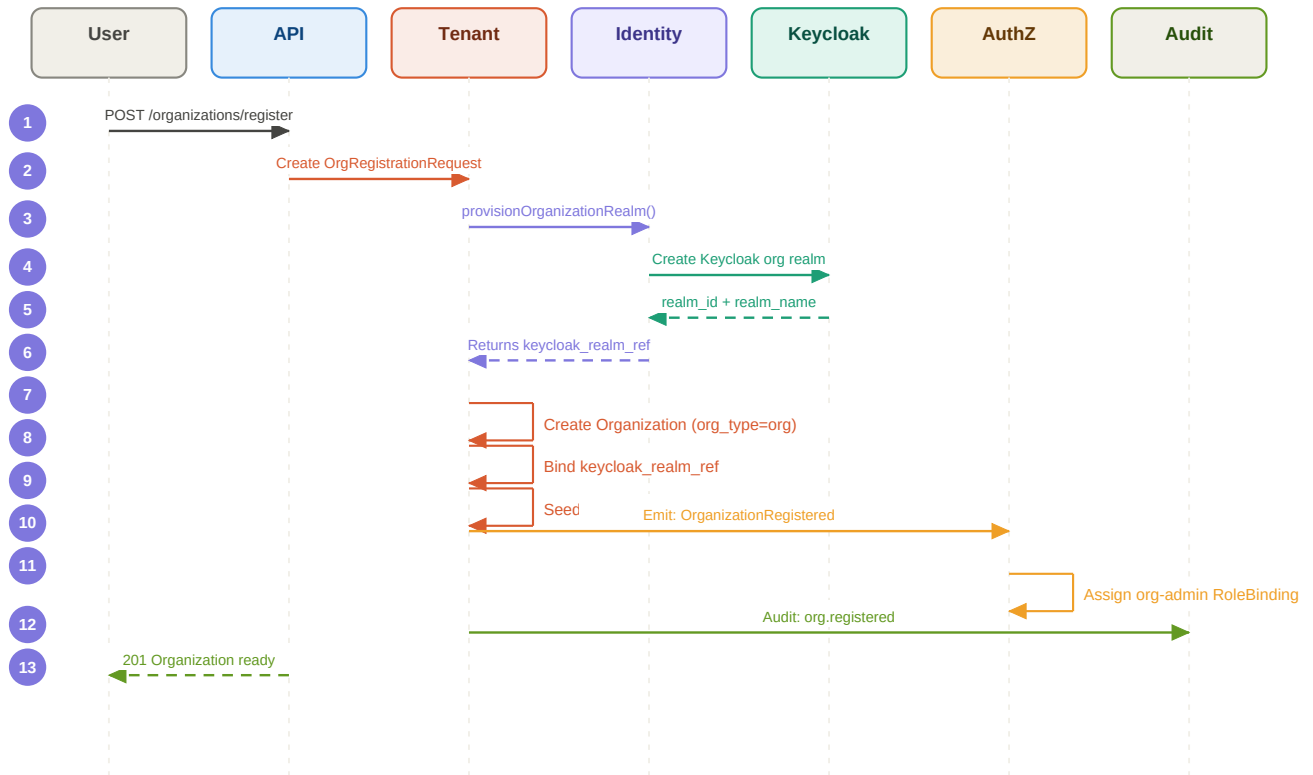


Figure 4 — Org registration: Tenant domain orchestrates, Identity domain creates the Keycloak realm

Example: Bob (founder) registers Acme Corp. Keycloak creates realm "acme" at acme.auth.platform.io. All Acme employees (carol@acme.com, dev@acme.com) now log in through acme.auth.platform.io – not the shared platform realm. The Organization record stores keycloak_realms_ref = "acme" permanently.

1.6 JWT Token — What It Contains and Why It Matters

After login, Identity issues a **JWT (JSON Web Token)**. This is a signed, time-limited token that proves identity for 15 minutes. Every API call sends it in the Authorization header.

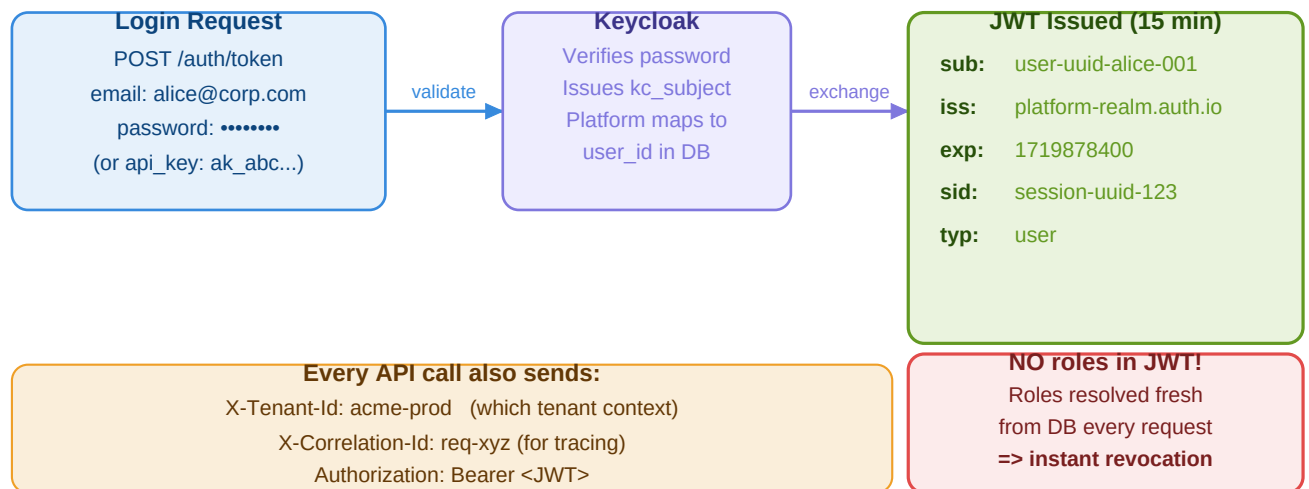


Figure 5 — JWT contents, where roles are NOT stored, and mandatory request headers

JWT Claim	What it means	Example value
sub	Keycloak subject — resolved to user_id via Credential table	kc-sub-a8f3b2c1
iss	Issuer — which realm generated this token	acme.auth.platform.io
exp	Expiry — token invalid after this timestamp	1719878400 (15 min)
sid	Session ID — links to Session record for revocation	session-uuid-123
typ	Principal type: user or sa (service account)	user

■ **CRITICAL:** Roles and permissions are NEVER put in the JWT. They are queried fresh from the database on every single API request using the user_id and X-Tenant-Id header. This means revoking access takes effect on the very next API call — no waiting for the token to expire.

PART 2 — AUTHORIZATION DOMAIN

The Authorization domain answers: **"Is this authenticated person allowed to do this specific thing in this specific place?"** It runs after Identity confirms who you are. It uses **RBAC (Role-Based Access Control)** — permissions bundled into roles, roles assigned to users at a specific scope.

2.1 RBAC — The Core Concept

Instead of assigning permissions to each person individually (which would be chaotic with thousands of users), RBAC bundles permissions into named roles, then assigns roles to users. Think of it like job titles in a company.

Bank analogy: "Manager" role = can approve loans + view all accounts + add staff.
"Teller" role = can view accounts + process transactions only. Define the roles **ONCE**.
Assign to many people. Change the role definition and everyone with that role is updated immediately.

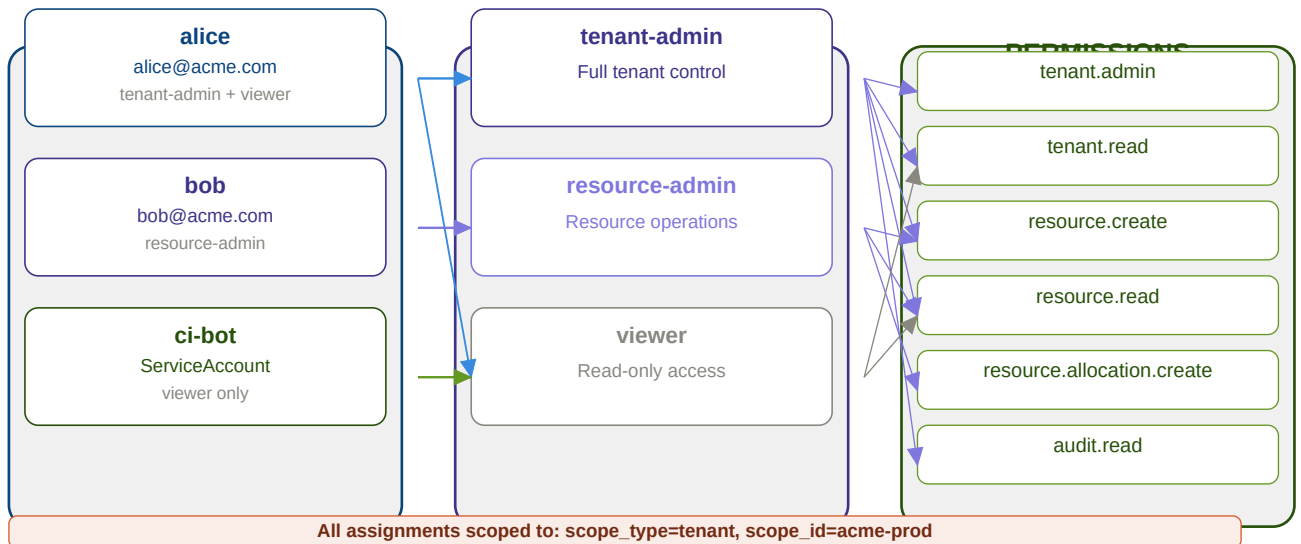


Figure 6 — RBAC chain: User gets Role, Role has Permissions. Scope = acme-prod tenant.

Table	Stores	Uniqueness rule
permissions	All possible action codes (global catalog)	code is globally unique
roles	Named bundles of permissions — tenant or platform scoped	(scope_type, scope_id, name) unique
role_permissions	Which permissions belong to each role	(role_id, permission_id) unique
user_roles	Which user has which role at which scope	(scope_type, scope_id, user_id, role_id) unique

2.2 Two Scopes — The Most Important Concept

Every role and permission belongs to exactly one of two completely separate scopes. They **never mix**. A customer role can never grant operator access. An operator role never appears in customer API checks.

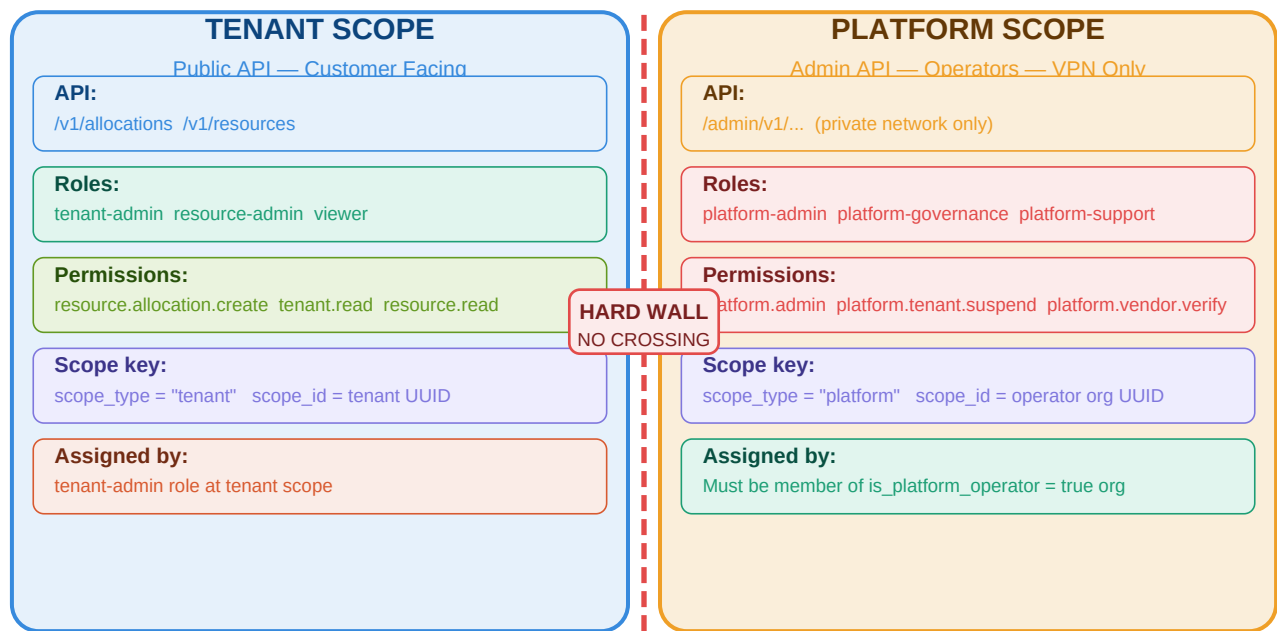


Figure 7 — Tenant scope (left) vs Platform scope (right) — separated by a hard wall

Property	Tenant Scope	Platform Scope
API surface	Public — /v1/...	Admin only — /admin/v1/... via VPN
Who gets it	Any active tenant member	Only is_platform_operator=true org members
scope_type	"tenant"	"platform"
scope_id	Tenant UUID (e.g. acme-prod-id)	Platform operator org UUID
Example role	tenant-admin, resource-admin, viewer	platform-admin, platform-support
Example perm	resource.allocation.create	platform.tenant.suspend

2.3 Permission Catalog

Tenant-Scoped Permissions

Permission Code	What it allows	Typical role
tenant.read	View tenant metadata and settings	viewer, resource-admin, tenant-admin
tenant.admin	Manage members, roles, settings	tenant-admin only
resource.read	List and read resource instances	viewer and above
resource.create	Create new resource records	resource-admin, tenant-admin
resource.delete	Delete resource records	resource-admin, tenant-admin
resource.allocation.create	Submit an allocation request	resource-admin, tenant-admin

Permission Code	What it allows	Typical role
resource.allocation.approve	Approve a pending allocation	tenant-admin only
resource.allocation.release	Release / decommission a resource	resource-admin, tenant-admin
audit.read	Read audit logs for this tenant	tenant-admin only

Platform-Scoped Permissions (Admin API Only)

Permission Code	What it allows	Typical role
platform.admin	Full platform configuration	platform-admin
platform.tenant.suspend	Suspend any customer tenant	platform-admin, security team
platform.tenant.read	Read any tenant data (support)	platform-support
platform.vendor.verify	Verify a vendor identity	platform-governance
platform.plugin.approve	Approve a plugin version for marketplace	platform-governance
platform.audit.read	Read audit logs across all tenants	platform-admin, compliance
platform.support	Read-only support bundle access	platform-support

2.4 Seeded Roles — Created Automatically

Per Tenant (seeded when OnboardingRequest completes)

Role	Purpose	Permissions included
tenant-admin	Full tenant control	All tenant.* + resource.* + audit.read
resource-admin	Resource operations	resource.* + resource.allocation.*
viewer	Read-only — monitoring	tenant.read + resource.read only

Acme Corp onboards. Three roles auto-created for acme-prod: tenant-admin: assigned to Bob (org owner) immediately. resource-admin: Alice requests and gets this from Bob. viewer: ci-bot service account assigned this for read-only monitoring.

Platform Operator Roles (seeded at bootstrap)

Role	Purpose	Permissions included
platform-admin	Full platform operations	platform.admin + platform.audit.read + platform.tenant.*
platform-governance	Marketplace governance	platform.vendor.verify + platform.plugin.approve
platform-support	Read-only customer support	platform.tenant.read + platform.support

2.5 Authorization Evaluation — Step by Step

Every single API request goes through this check. The platform asks: "Does user U have permission P in tenant T?" Steps run in order. First failure stops the evaluation.

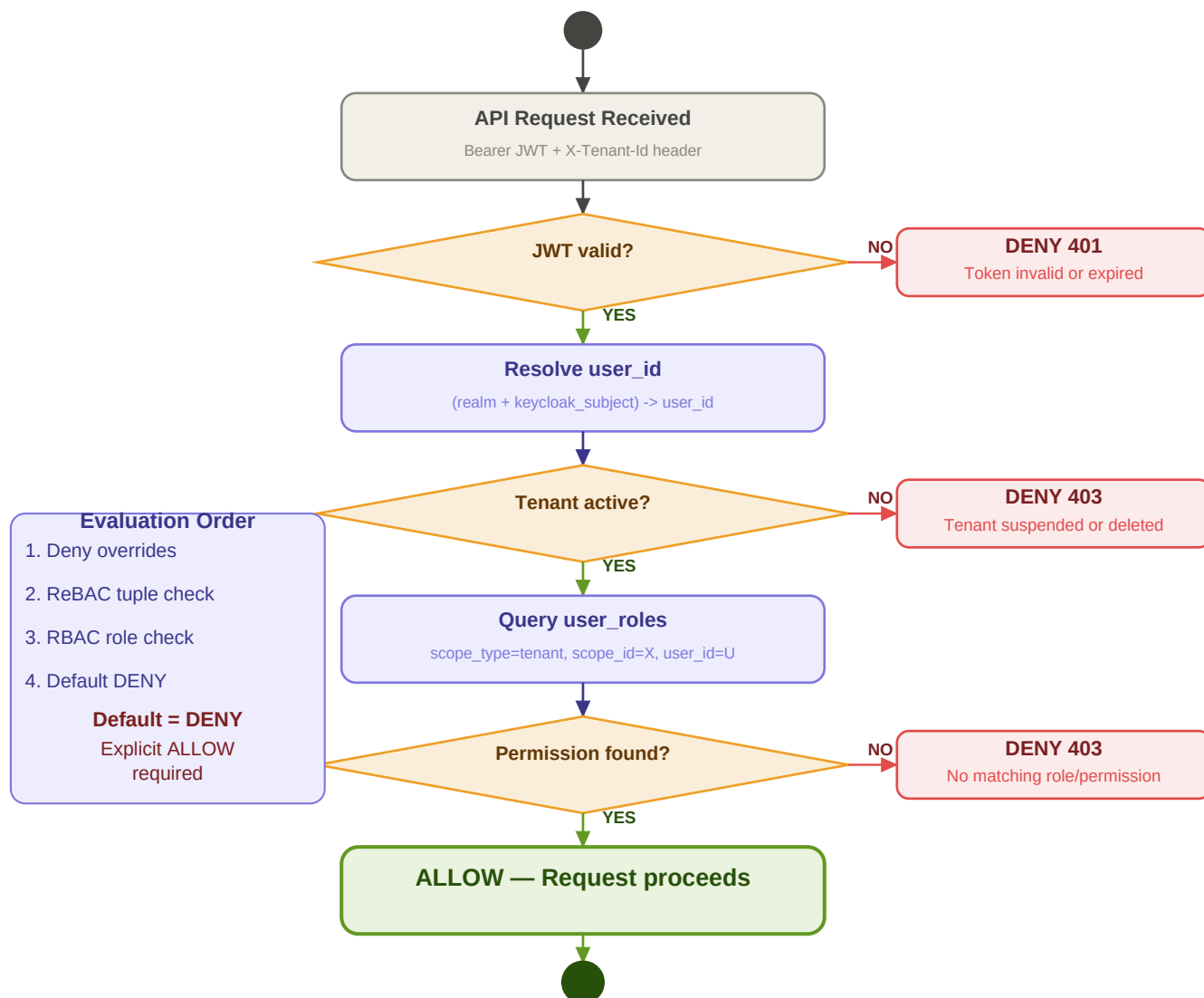


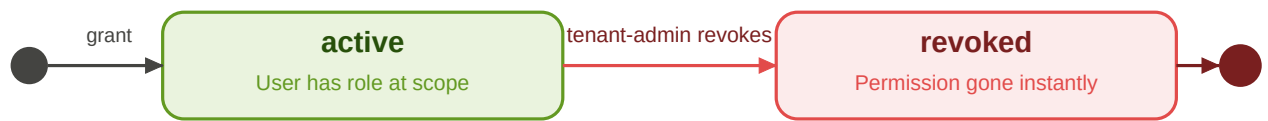
Figure 8 — Authorization evaluation flowchart: 3 decision points, default = DENY

Step	Check	Pass	Fail
1	JWT signature valid + not expired	Continue	401 Unauthorized
2	Resolve user_id from realm + kc_subject	user_id found	401 User not found
3	Is the tenant active?	Continue	403 Tenant not accessible
4	User has active TenantMembership?	Continue	403 Not a member
5	Query user_roles for this scope	Role IDs found	403 No roles
6	Do roles include required permission?	ALLOW	403 Permission denied

Concrete example – Alice calls POST /projects/payments/allocations: Step 1: JWT valid, iss=acme.auth.platform.io OK Step 2: kc-sub-a8f3b2c1 -> user-uuid-alice-001 OK Step 3: Tenant acme-prod status=active OK Step 4: Alice has active TenantMembership in acme-prod OK Step 5: user_roles -> resource-admin role in acme-prod OK Step 6: resource-admin includes resource.allocation.create ALLOW

2.6 RoleBinding — State Machine and Instant Revocation

A RoleBinding connects one user to one role at one scope. It is the most frequently created and revoked entity in the Authorization domain. Revocation is **instant** because roles are never stored in the JWT.



KEY RULE: Roles are NOT stored in the JWT.

When a RoleBinding is revoked, the DB row is deleted immediately.

The VERY NEXT API call by that user gets DENIED — no waiting for token to expire.

Example: Alice revoked at 10:00am. Her 10:01am call = DENY, even if JWT expires at 10:15am.

Figure 9 — RoleBinding states: active -> revoked. Revocation takes effect on the very next API call.

State	Meaning	Effect on API calls
active	User has this role — all role permissions granted	Allowed (subject to other checks)
revoked	Role removed — DB row deleted or soft-deleted	Next API call needing this role: DENIED

2.7 Security Rules — Never Break These

Rule	Why	Risk if broken
No roles in JWT	Revocation is instant	Revoked users keep access until JWT expires
Always include tenant_id in RBAC queries	Prevents cross-tenant role leakage	Alice's acme-dev admin role affects acme-prod
Platform API rejects customer tokens	Token issuer validated at ingress	Customer hits admin routes accidentally
Only tenant-admin can assign roles	Prevents privilege escalation	Any user gives themselves admin
Roles are never hard-deleted	Audit trail must show historical access	Cannot reconstruct permissions at incident time

2.8 Worked Examples

Example 1 — Alice approves an allocation (success path)

Step	What happens	Domain
1	Alice: POST /allocations/alloc-789/approve	API Gateway
2	JWT valid, iss=acme.auth.platform.io	Identity
3	Tenant acme-prod is active	Tenant
4	Check: resource.allocation.approve in acme-prod for alice	Authorization
5	alice has tenant-admin -> tenant-admin has that permission	Authorization
6	ALLOW: AllocationRequest -> approved state	Resource
7	AuditLog: allocation.approved by user-uuid-alice-001	Platform Core

Example 2 — Bob tries admin API with customer token (blocked)

Step	What happens	Result
1	Bob: GET /admin/v1/tenants with JWT from acme.auth.platform.io	Request received
2	Admin API middleware checks: iss must equal operator realm	iss = acme realm (WRONG)
3	Rejected at ingress — never reaches RBAC DB check	DENY 403
4	Even if DB has platform.tenant.read for bob — does not matter	Token level block

Example 3 — ci-bot service account creates a resource

Step	What happens	Domain
1	ci-bot sends API key: ak_abc12345_xyz... in X-API-Key header	API Gateway
2	Platform hashes received key, matches stored key_hash for sa	Identity
3	ServiceAccount active, tenant_id=acme-prod confirmed	Identity + Tenant
4	Check: resource.create in acme-prod for sa-deploy-prod	Authorization
5	sa has resource-admin -> resource-admin includes resource.create	Authorization
6	ALLOW: resource creation proceeds	Resource

End of document — Identity & Authorization Module Reference (v2)

Next: Resource Domain — quota enforcement and allocation state machine